

BDA - Monitorización de un clúster distribuido con Grafana

Alberto Piedra Venegas

April 2026

Índice

1. Instalación y configuración de herramientas de monitorización	2
1.1. Instalación Node Exporter	2
1.2. Instalación Prometheus	3
1.3. Instalación Grafana	5
2. Recogida, procesamiento y visualización de métricas	6
2.1. Visualización: Uso de CPU del nodo	7
2.2. Visualización: Tráfico de red	9
2.3. Visualización: Memoria RAM usada	10
2.4. Visualización: Ocupación del disco	10
3. Generación de alertas ante riesgos o mal funcionamiento	11
3.1. Alerta: Uso del disco alto	11
3.2. Alerta: Uso del disco alto	14
3.3. Alerta: Uso de memoria RAM alta	15
3.4. Alerta: Tráfico de red crítico	16
4. Evaluación del rendimiento de la solución de la monitorización	16
5. Análisis de la estabilidad del sistema y servicios monitorizados	17

1. Instalación y configuración de herramientas de monitorización

1.1. Instalación Node Exporter

Lo primero que haremos será la descarga del **Node exporter**, esta herramienta se encarga de recopilar metricas de hardware y del sistema operativo (CPU, memoria, disco, etc) y exponerlas a un endpoint para que la herramienta de Prometheus pueda acceder a ella.

```
hadoop@nodo1:~$ wget https://github.com/prometheus/node_exporter/releases/download/v1.10.2/node_exporter-1.10.2.linux-amd64.tar.gz
```

Extraemos el contenido de la descarga, y lo enviamos a nuestros otros dos nodos.

```
hadoop@nodo1:~$ tar xvzf node_exporter-1.10.2.linux-amd64.tar.gz
```

```
hadoop@nodo1:~$ scp -r node_exporter-1.10.2.linux-amd64/ hadoop@nodo2:/home/hadoop
hadoop@nodo1:~$ scp -r node_exporter-1.10.2.linux-amd64/ hadoop@nodo3:/home/hadoop
```

Ya podriamos ejecutar la herramienta y comprobar que funcione.

```
hadoop@nodo1:~$ cd node_exporter-1.10.2.linux-amd64/
hadoop@nodo1:~/node_exporter-1.10.2.linux-amd64$ ls
LICENSE  node_exporter  NOTICE
hadoop@nodo1:~/node_exporter-1.10.2.linux-amd64$ ./node_exporter &
```

```
localhost:9100/metrics x +
← → ↻ http://localhost:9100/metrics

# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
go_gc_duration_seconds_count 0
# HELP go_gc_pgc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the G0GC environment variable, and the runtime/debug.SetGCPercent function. Sourced from /gc/pgc/percent.
go_gc_pgc_percent 100
# HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environment variable, and the runtime/debug.SetMemoryLimit function. Sourced from /gc/gomemlimit/bytes
go_gc_gomemlimit_bytes 9.223372036854776e+18
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 7
# HELP go_info Information about the Go environment.
# TYPE go_info gauge{go:"1.25.3"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 1.613112e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 1.613112e+06
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.445339e+06
# HELP go_memstats_frees_total Total number of heap objects freed. Equals to /gc/heap/frees:objects + /gc/heap/tiny/allocs:objects.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 685
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata. Equals to /memory/classes/metadata/other:bytes.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 2.841356e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and currently in use, same as go_memstats_alloc_bytes. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 1.613112e+06
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used. Equals to /memory/classes/heap/released:bytes + /memory/classes/heap/free:bytes.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 1.541952e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use. Equals to /memory/classes/heap/objects:bytes + /memory/classes/heap/unused:bytes
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 2.12851e+06
# HELP go_memstats_heap_objects Number of currently allocated objects. Equals to /gc/heap/objects:objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 8138
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS. Equals to /memory/classes/heap/released:bytes.
# TYPE go_memstats_heap_released_bytes gauge
go_memstats_heap_released_bytes 1.541952e+06
# HELP go_memstats_heap_sys_bytes Number of heap bytes obtained from system. Equals to /memory/classes/heap/objects:bytes + /memory/classes/heap/unused:bytes + /memory/classes/heap/free:bytes.
# TYPE go_memstats_heap_sys_bytes gauge
go_memstats_heap_sys_bytes 1.803888e+06
# HELP go_memstats_last_gc_time_seconds Number of seconds since 1970 of last garbage collection.
# TYPE go_memstats_last_gc_time_seconds gauge
go_memstats_last_gc_time_seconds 0
# HELP go_memstats_mallocs_total Total number of heap objects allocated, both live and gc-ed. Semantically a counter version for go_memstats_heap_objects gauge. Equals to /gc/heap/allocs:objects + /gc/heap/tiny/allocs:objects.
# TYPE go_memstats_mallocs_total counter
go_memstats_mallocs_total 8834
# HELP go_memstats_mcache_inuse_bytes Number of bytes in use by mcache structures. Equals to /memory/classes/metadata/mcache/inuse:bytes.
# TYPE go_memstats_mcache_inuse_bytes gauge
go_memstats_mcache_inuse_bytes 1288
# HELP go_memstats_mcache_sys_bytes Number of bytes used for mcache structures obtained from system. Equals to /memory/classes/metadata/mcache/inuse:bytes + /memory/classes/metadata/mcache/free:bytes.
# TYPE go_memstats_mcache_sys_bytes gauge
go_memstats_mcache_sys_bytes 12584
# HELP go_memstats_mspan_inuse_bytes Number of bytes in use by mspan structures. Equals to /memory/classes/metadata/mspan/inuse:bytes.
# TYPE go_memstats_mspan_inuse_bytes gauge
go_memstats_mspan_inuse_bytes 3368
# HELP go_memstats_mspan_sys_bytes Number of bytes used for mspan structures obtained from system. Equals to /memory/classes/metadata/mspan/inuse:bytes + /memory/classes/metadata/mspan/free:bytes.
# TYPE go_memstats_mspan_sys_bytes gauge
go_memstats_mspan_sys_bytes 3368
# HELP go_memstats_next_gc_bytes Number of heap bytes when next garbage collection will take place. Equals to /gc/heap/goal:bytes.
# TYPE go_memstats_next_gc_bytes gauge
go_memstats_next_gc_bytes 4.184266e+06
# HELP go_memstats_other_sys_bytes Number of bytes used for other system allocations. Equals to /memory/classes/other:bytes.
```

Comprobaremos que el endpoint **metrics** funcione y que de información.

```
localhost:9100/metrics x nodo2:9100/metrics x +
← → ↻ No seguro http://nodo2:9100/metrics

# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
go_gc_duration_seconds_count 0
# HELP go_gc_pgc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the G0GC environment variable, and the runtime/debug.SetGCPercent function. Sourced from /gc/pgc/percent.
go_gc_pgc_percent 100
# HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environment variable, and the runtime/debug.SetMemoryLimit function. Sourced from /gc/gomemlimit/bytes
go_gc_gomemlimit_bytes 9.223372036854776e+18
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 7
# HELP go_info Information about the Go environment.
# TYPE go_info gauge{go:"1.25.3"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 966168
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 966168
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.444379e+06
# HELP go_memstats_frees_total Total number of heap objects freed. Equals to /gc/heap/frees:objects + /gc/heap/tiny/allocs:objects.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 685
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata. Equals to /memory/classes/metadata/other:bytes.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 2.848335e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and currently in use, same as go_memstats_alloc_bytes. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 966168
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used. Equals to /memory/classes/heap/released:bytes + /memory/classes/heap/free:bytes.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 1.613112e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use. Equals to /memory/classes/heap/objects:bytes + /memory/classes/heap/unused:bytes
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 2.429632e+06
# HELP go_memstats_heap_objects Number of currently allocated objects. Equals to /gc/heap/objects:objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 8086
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS. Equals to /memory/classes/heap/released:bytes.
# TYPE go_memstats_heap_released_bytes gauge
go_memstats_heap_released_bytes 1.641952e+06
```

1.2. Instalación Prometheus

Seguiremos con la instalación de **Prometheus**, el cual se encargará de recopilar las metricas en tiempo real almacenandolas en series temporales dentro de una base de datos.

Para que la herramienta de Prometheus pueda acceder a la información de los 3 nodos, será necesario modificar el siguiente archivo:

```
hadoop@nodo1:~$ sudo nano /etc/prometheus/prometheus.yml
```

Dentro de este archivo añadiremos nuestros nodos para que pueda detectarlos.

```
alerting:
  alertmanagers:
    - static_configs:
      - targets: ['localhost:9093']

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label 'job=<job_name>' to any timeseries scraped from this config.
  - job_name: 'prometheus'

    # Override the global default and scrape targets from this job every 5 seconds.
    scrape_interval: 5s
    scrape_timeout: 5s

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

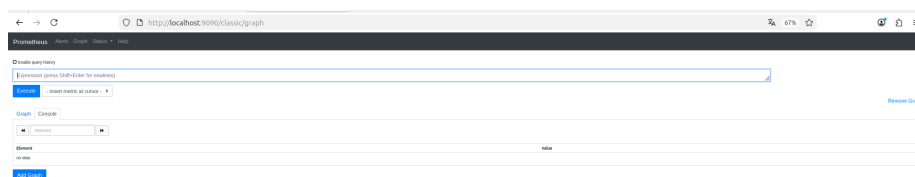
    static_configs:
      - targets: ['localhost:9090']

  - job_name: node
    # If prometheus-node-exporter is installed, grab stats about the local
    # machine by default.
    static_configs:
      - targets: ['localhost:9100', 'nodo2:9100', 'nodo3:9100']
```

Reiniciaremos el servicio, para que se guarden los cambios que hemos realizado.

```
hadoop@nodo1:~$ sudo systemctl restart prometheus
```

Y comprobaremos en el navegador que se ha instalado correctamente.



De la misma manera comprobaremos que Prometheus detecta los 3 nodos que tenemos.

Endpoint	State	Labels	Last Scrape	Sample Duration Error
http://localhost:9100/metrics	UP	instance="localhost:9100" job="prometheus"	9.827s ago	460.0ms
http://localhost:9100/metrics	UP	instance="localhost:9100" job="prometheus"	9.828s ago	467.0ms
http://localhost:9100/metrics	UP	instance="localhost:9100" job="prometheus"	7.08s ago	1.117s

Endpoint	State	Labels	Last Scrape	Sample Duration Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	1.238s ago	18.07ms

1.3. Instalación Grafana

Ahora seguiremos con la instalación de Grafana mediante los siguientes comandos.

```
Procesando disparadores para desktop-file-utils (0.27-2build1) ...
hadoop@nodo1:~$ sudo apt install -y apt-transport-https software-properties-common
```

```
hadoop@nodo1:~$ sudo add-apt-repository "deb https://packages.grafana.com/oss/release/debian stable main"
```

```
hadoop@nodo1:~$ wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
```

```
hadoop@nodo1:~$ wget https://dl.grafana.com/oss/release/grafana_11.5.1_amd64.deb
--2026-03-25 17:10:45-- https://dl.grafana.com/oss/release/grafana_11.5.1_amd64.deb
Resolviendo dl.grafana.com (dl.grafana.com)... 151.101.134.217, 2a04:4e42:1f::729
Conectando con dl.grafana.com (dl.grafana.com)[151.101.134.217]:443... conectado.
Petición HTTP enviada, esperando respuesta... 200 OK
Longitud: 138745442 (132M) [application/octet-stream]
Guardando como: 'grafana_11.5.1_amd64.deb'

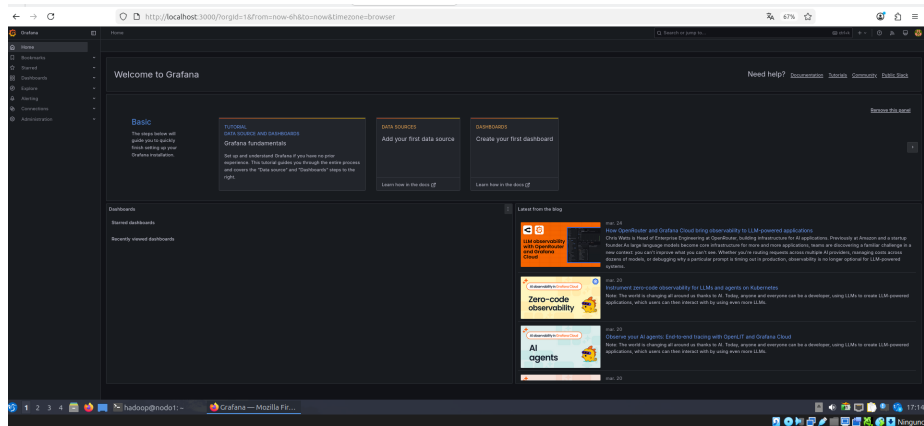
grafana_11.5.1_amd64.deb          1%[>
```

```
hadoop@nodo1:~$ sudo apt install ./grafana_11.5.1_amd64.deb -y
```

Con toda la instalación realizada, lo siguiente que haremos será iniciar la herramienta de Grafana y habilitarla.

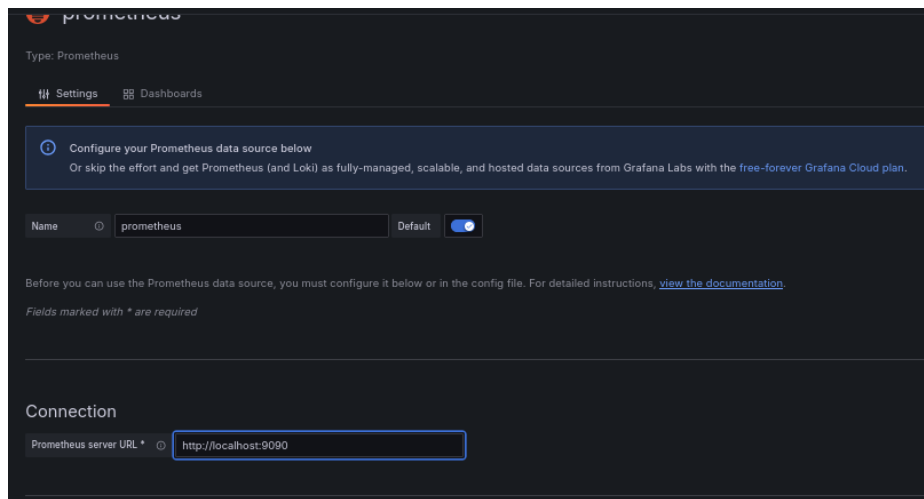
```
hadoop@nodo1:~$ sudo systemctl start grafana-server
hadoop@nodo1:~$ sudo systemctl enable grafana-server
Synchronizing state of grafana-server.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable grafana-server
Created symlink /etc/systemd/system/multi-user.target.wants/grafana-server.service → /usr/lib/systemd/system/grafana-server.service.
```

Con esto ya tendríamos instalada la herramienta de Grafana y podríamos comprobarlo a través de su interfaz web.



2. Recogida, procesamiento y visualización de métricas

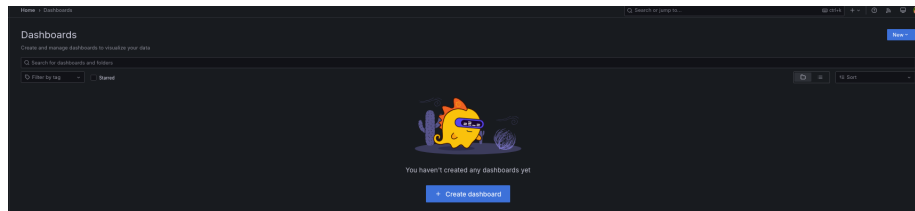
Lo primero que tendremos que hacer es realizar la conexión entre Grafana y Prometheus para ello, añadiremos un **Data Source**.



Comprobamos que la conexión se realiza correctamente.

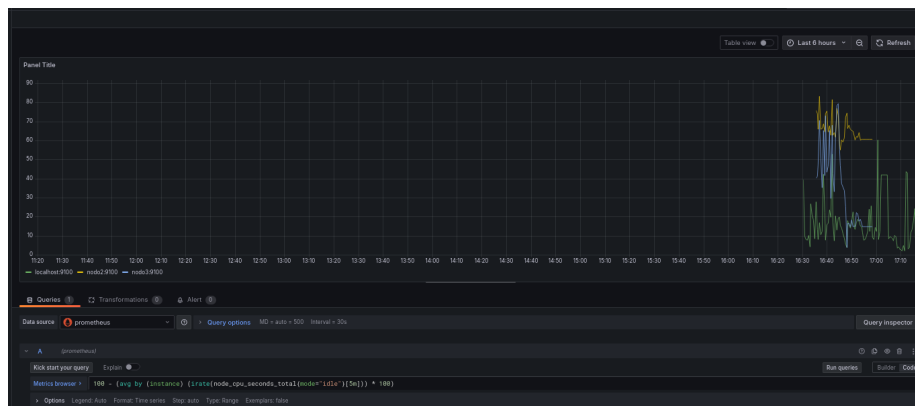


Lo siguiente que tendremos que hacer será crear el dashboard donde tendremos nuestras visualizaciones que nos mostrarán información acerca del clúster.



Crearemos las visualizaciones para nuestro dashboard, la primera será el uso de la CPU de cada nodo.

2.1. Visualización: Uso de CPU del nodo



También modificaremos algunas características para mejorar el entendimiento de la visualización, en este caso cambiaremos la estandarización de las unidades para que salgan como porcentajes.

 Time series ▼

 Search options

All Overrides

Solid Dash Dots

Connect null values

Never Always Threshold

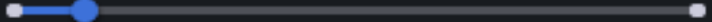
Disconnect values

Never Threshold

Show points

Auto Always Never

Point size

 1 40 5

Stack series

Off Normal 100%

Standard options ▼

Unit

Percent (0-100) × ▼

Min

Leave empty to calculate based on all values

auto

Max

Leave empty to calculate based on all values

auto

Field min/max

Calculate min max per field

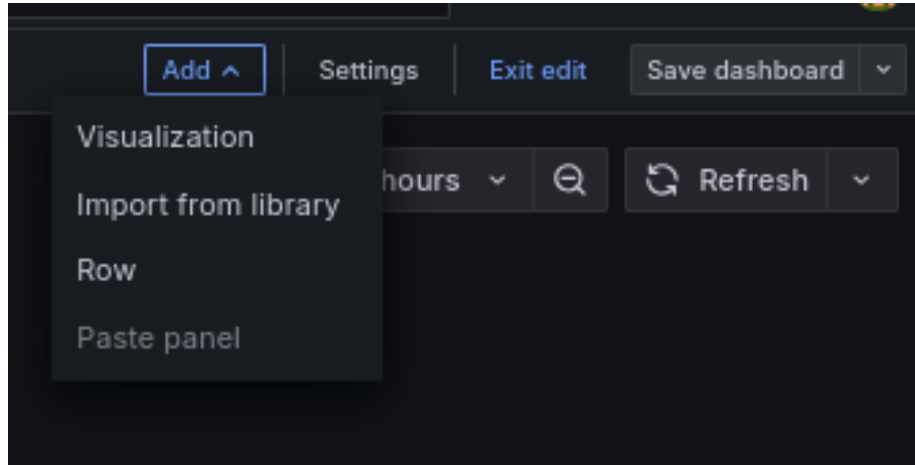
☐

Decimals

auto

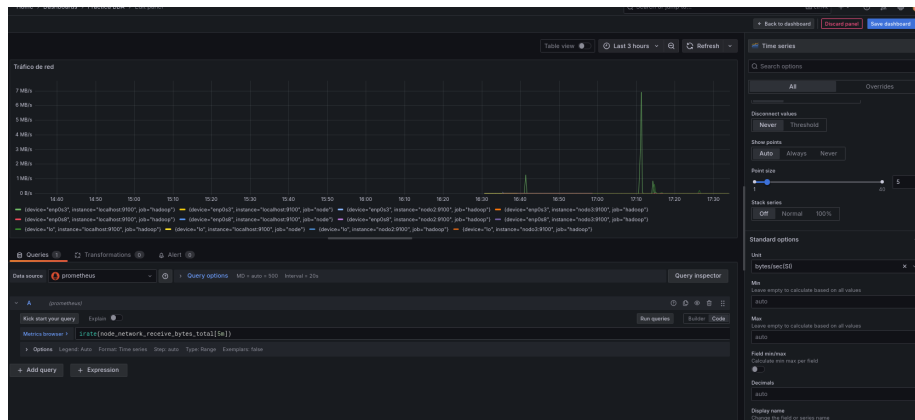
Display name

Por último añadiremos esta visualización a nuestro dashboard.

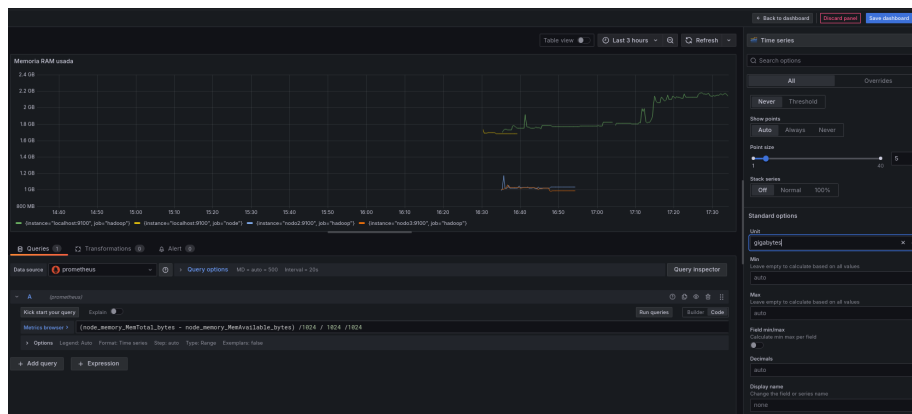


Realizaremos las otras visualizaciones de la misma manera:

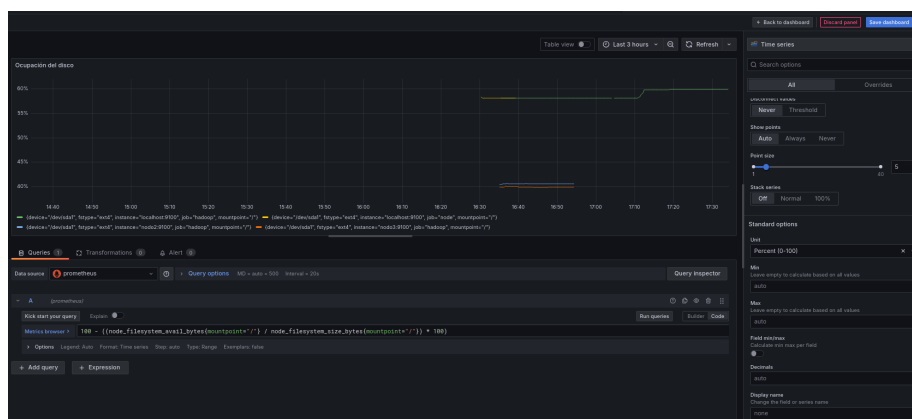
2.2. Visualización: Trafico de red



2.3. Visualización: Memoria RAM usada



2.4. Visualización: Ocupación del disco



Con todas nuestras visualizaciones creadas, ya podremos añadirlas al dashboard.



3. Generación de alertas ante riesgos o mal funcionamiento

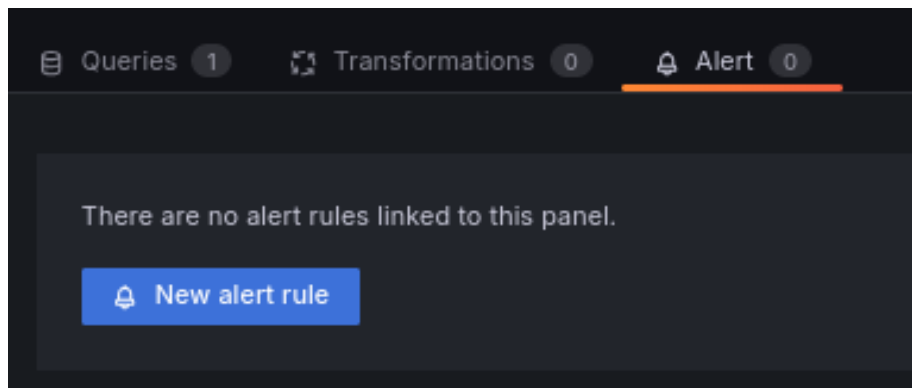
Para poder evitar que ocurran problemas que afecten al cluster sin que nos demos cuenta, podremos añadir alertas para que nos notifique si alguna métrica alcanza un punto de riesgo.

Para poder crear estas alertas entraremos en una visualización y pulsaremos en el botón de editar.

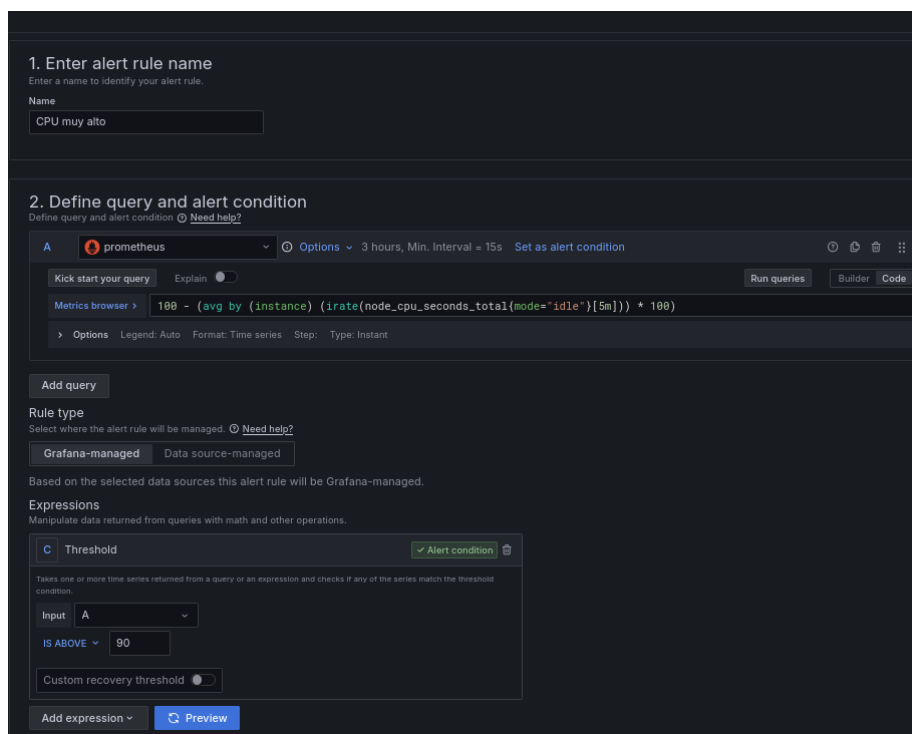
3.1. Alerta: Uso del disco alto



Aquí dentro crearemos la alerta



Primero elegiremos la metrica que queremos supervisar y el umbral, que debe sobrepasar para que la alerta se dispare.



Le añadiremos un punto de contacto, en este caso será un correo electronico al que nos notificarán si esa alerta se dispara.

Contact points

Choose how to notify your contact points when an alert instance fires

Create contact point

Name *

Name

Integration

Email

Test Duplicate Delete

Addresses

You can enter multiple email addresses using a ",", "n" or "+" separator

albertopiedra155@gmail.com

Optional Email settings

Notification settings

+ Add contact point integration

Save contact point Cancel

Por último crearemos una carpeta donde guardaremos todas las alertas del dashboard.

3. Add folder and labels

Organize your alert rule with a folder and set of labels. [Need help?](#)

Folder

Select a folder to store your rule in.

HadoopAlertas or + New folder

Labels

Add labels to your rule for searching, silencing, or routing to a notification policy. [Need help?](#)

No labels selected + Add labels

4. Set evaluation behavior

Define how the alert rule is evaluated. [Need help?](#)

Evaluation group and interval

Select an evaluation group... or + New evaluation group

Pending period

Period during which the threshold condition must be met to trigger an alert. Selecting "None" triggers the alert immediately once the condition is met.

1m

None 1m 2m 3m 4m 5m

> Configure no data and error handling

5. Configure notifications

Select who should receive a notification when an alert rule fires.

Recipient

Select contact point Use notification policy

Notifications for firing alerts are routed to a selected contact point. [Need help?](#)

Alertmanager: grafana

Contact point

EmailAlberto View or create contact points

Email albertopiedra155@gmail.com

Y ya por fin tendríamos la alerta creada.

Alert			
State	Name	Health	Actions
Normal	CPU muy alto	ok	View Edit More

Lo siguiente que haremos será crear las alertas para las demás visualizaciones.

3.2. Alerta: Uso del disco alto

Name

Uso del disco alto

2. Define query and alert condition

Define query and alert condition [Need help?](#)

A

prometheus

Options

3 hours, Min. Interval = 15s

Set as alert condition

Kick start your query

Explain

Run queries

Builder

Code

Metrics browser

100 - (((node_filesystem_avail_bytes{mountpoint="/"}) / node_filesystem_size_bytes{mountpoint="/"}) * 100)

Options

Legend: Auto

Format: Time series

Step: Type: Instant

Add query

Rule type

Select where the alert rule will be managed. [Need help?](#)

Grafana-managed

Data source-managed

Based on the selected data sources this alert rule will be Grafana-managed.

Expressions

Manipulate data returned from queries with math and other operations.

C

Threshold

Alert condition

Takes one or more time series returned from a query or an expression and checks if any of the series match the threshold condition.

Input

A

IS ABOVE

90

Custom recovery threshold

Add expression

Preview

14

3.3. Alerta: Uso de memoria RAM alta

New alert rule

1. Enter alert rule name

Enter a name to identify your alert rule.

Name

Memoria RAM usada alta

2. Define query and alert condition

Define query and alert condition. [Need help?](#)

A

 prometheus

Options 3 hours, Min. Interval = 15s [Set as alert condition](#)

Kick start your query

Explain

Run queries

Builder

Code

Metrics browser

(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / 1024 / 1024 / 1024

Options

Legend: Auto

Format: Time series

Step: Type: Instant

Add query

Rule type

Select where the alert rule will be managed. [Need help?](#)

Grafana-managed

Data source-managed

Based on the selected data sources this alert rule will be Grafana-managed.

Expressions

Manipulate data returned from queries with math and other operations.

C

Threshold

Alert condition

Takes one or more time series returned from a query or an expression and checks if any of the series match the threshold condition.

Input

A

IS ABOVE

3.5

Custom recovery threshold

Add expression

Preview

15

3.4. Alerta: Tráfico de red crítico

Edit rule

1. Enter alert rule name

Enter a name to identify your alert rule.

Name

Tráfico de red crítico

2. Define query and alert condition

Define query and alert condition [Need help?](#)

A

prometheus

Options

3 hours, MD = 43200, Min. Interval = 15s

Set as alert condition

⌵

⌵

⌵

Kick start your query

Explain

Run queries

Builder

Code

Metrics browser >

`irate(node_network_receive_bytes_total[5m])`

> Options

Legend: Auto

Format: Time series

Step: Type: Instant

Add query

Rule type

Select where the alert rule will be managed. [Need help?](#)

Grafana-managed

Data source-managed

The alert rule type cannot be changed for an existing rule.

Expressions

Manipulate data returned from queries with math and other operations.

C

Threshold

✓ Alert condition

⌵

Takes one or more time series returned from a query or an expression and checks if any of the series match the threshold condition.

Input

A

⌵

IS ABOVE

50

Custom recovery threshold

⌵

4. Evaluación del rendimiento de la solución de la monitorización

Para comprobar los recursos que consumen tanto Grafana como Prometheus utilizaremos el comando de linux **top**.

```
top - 18:10:28 up 1:03, 3 users, load average: 1,97, 2,01, 1,84
Tasks: 206 total, 2 ejecutar, 204 hibernar, 0 detener, 0 zombie
%Cpu(s): 3,3 us, 5,4 sy, 3,5 ni, 86,2 id, 0,1 wa, 0,0 hi, 1,5 si, 0,0 st
MiB Mem : 6682,1 total, 3667,6 libre, 1071,9 usado, 2193,2 búf/caché
MiB Intercambio: 512,0 total, 512,0 libre, 0,0 usado, 5610,3 dispon Mem
```

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	HORA+	ORDEN
2503	hadoop	20	0	21408	7296	6396	S	4,0	0,1	2:23.07	picom
3462	hadoop	20	0	1243168	21124	13844	S	4,0	0,3	0:01.71	node_exporter
3468	hadoop	20	0	17368	6276	4000	R	2,0	0,1	0:00.79	top

16

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	HORA+	ORDEN
48069	grafana	20	0	1647156	207060	132696	S	2,0	3,0	2:27.86	grafana
1359	mysql	20	0	2376416	398704	38016	S	1,0	5,8	5:59.18	mysqld
45861	prometh+	20	0	2080544	86452	34328	S	0,7	1,3	6:46.72	prometheus
23	root	rt	0	0	0	0	S	0,3	0,0	0:01.87	migration/1
26138	hadoop	20	0	471656	121244	96916	S	0,3	1,8	0:58.57	qterminal
38855	hadoop	20	0	2686692	180620	91620	S	0,3	2,6	0:47.94	Privileged Cont
48388	hadoop	20	0	2576448	71968	56680	S	0,3	1,1	0:05.34	Web Content
50501	hadoop	20	0	17372	6272	4000	R	0,3	0,1	0:00.02	top

Podremos comprobar que ambas herramientas no consumen una gran cantidad de recursos y no afectan al funcionamiento del nodo maestro.

5. Análisis de la estabilidad del sistema y servicios monitorizados

Para comprobar como se desempeña los nodos, ejecutaremos un trabajo de mapreduce de Hadoop y comprobaremos en las visualizaciones.

```
hadoop@nodo1: /opt/hadoop/share/hadoop/mapreduce$ hadoop jar hadoop-mapreduce-examples-3.4.1.jar pi 10 1000
```



Como se puede ver en la gráfica el uso de CPU está bastante alto mientras se ejecuta el trabajo y tiene unos momentos donde el uso es menor cuando activamos el comando `top` en cada nodo ya que deja de usar recursos en el trabajo mientras el comando esté en ejecución.